

Apache Spark Note Guide (Extended)

DataFrame API, Spark SQL, Transformations & Actions

1. The DataFrame API & Core Setup

Spark operates on a paradigm of **Lazy Evaluation**. Instructions are divided into **Transformations** (build the logic graph) and **Actions** (execute the graph). Below is the setup for Jupyter.

```
# Cell 1: Initialize Spark Session & Create Data
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, desc, sum, avg

# Initialize
spark = SparkSession.builder.appName("SparkCompleteNotes").getOrCreate()

# Create Base DataFrame
data = [
    (1, "Alice", "Engineering", 75000, 25),
    (2, "Bob", "Marketing", 60000, 30),
    (3, "Charlie", "Engineering", 80000, 35),
    (4, "David", "Sales", 65000, 28),
    (5, "Eve", "Marketing", 70000, 32)
]
columns = ["Id", "Name", "Department", "Salary", "Age"]
df = spark.createDataFrame(data, columns)
```

2. All Key Transformations (Lazy)

Transformations return a new DataFrame. They are not executed until an action is called.

```

# 1. select(): Choose specific columns
df_select = df.select("Name", "Department")

# 2. filter() / where(): Apply conditions
df_filter = df.filter(col("Salary") > 65000)
# Or: df.where(col("Department") == "Engineering")

# 3. withColumn(): Add or modify a column
df_with_col = df.withColumn("Bonus", col("Salary") * 0.10)

# 4. withColumnRenamed(): Rename a column
df_renamed = df.withColumnRenamed("Id", "Emp_Id")

# 5. drop(): Remove a column
df_drop = df.drop("Age")

# 6. orderBy() / sort(): Sort the data
df_sorted = df.orderBy(desc("Salary"))

# 7. groupBy(): Aggregate data
df_grouped = df.groupBy("Department").agg(sum("Salary").alias("Total_Salary"))

# 8. join(): Combine two DataFrames
data_dept = [("Engineering", "Building A"), ("Marketing", "Building B")]
df_dept = spark.createDataFrame(data_dept, ["Department", "Location"])

df_joined = df.join(df_dept, on="Department", how="inner")

```

3. All Key Actions (Eager)

Actions trigger the DAG execution.

```

# 1. show(): Print rows to the console
df_joined.show()

# 2. count(): Get the number of rows
total_rows = df.count()

# 3. collect(): Return all rows to the driver as a list of Row objects (Caution: High Memory Usage)
all_data_list = df.collect()

# 4. first() / take(n): Return the first row, or first N rows
first_row = df.first()
first_three = df.take(3)

# 5. write(): Save DataFrame to disk
# df.write.mode("overwrite").parquet("output/employees.parquet")
# df.write.mode("overwrite").csv("output/employees.csv", header=True)

```

4. Spark SQL

Execute standard SQL directly on your distributed data.

```
# Register Temp View
df.createOrReplaceTempView("employees")

# Query with standard SQL
sql_df = spark.sql('''
    SELECT Department, AVG(Salary) as Avg_Salary
    FROM employees
    GROUP BY Department
    ORDER BY Avg_Salary DESC
''')
sql_df.show()
```

5. Docker Guide: Running Jupyter Notebook as a Terminal Script

If you want to write a Jupyter notebook (`app.ipynb`) and then have Docker build an image and execute that notebook headlessly (printing the output to the terminal), follow these steps:

Step 1: Create your Jupyter Notebook (`app.ipynb`)

Create a basic notebook in your local Jupyter Lab environment, add the PySpark code from above into the cells, and save it as `app.ipynb` in your project folder.

Step 2: Create the Dockerfile

In the exact same directory as your notebook, create a file named `Dockerfile` (no extension):

```
# Use an official Python runtime
FROM python:3.10-slim

# Install Java (Required for Apache Spark)
RUN apt-get update && apt-get install -y default-jre && rm -rf /var/lib/apt/lists/*

# Install PySpark and Jupyter
RUN pip install --no-cache-dir pyspark jupyter nbconvert

# Set the working directory
WORKDIR /app

# Copy the Jupyter notebook into the container
COPY app.ipynb .

# CMD to execute the notebook headlessly and output the text directly to the terminal!
CMD ["jupyter", "nbconvert", "--execute", "--to", "markdown", "app.ipynb", "--stdout"]
```

Step 3: Build and Run

Run these commands in your terminal where the Dockerfile is located:

```
# 1. Build the Docker Image (name it 'spark-headless')
docker build -t spark-headless .

# 2. Run the Container
docker run --rm spark-headless
```

Result: Docker will boot, Java and Spark will initialize, your `app.ipynb` will execute cell by cell, and the printed tables (like `df.show()`) will be generated directly in your terminal output!